计 算 机 体 系 结 构

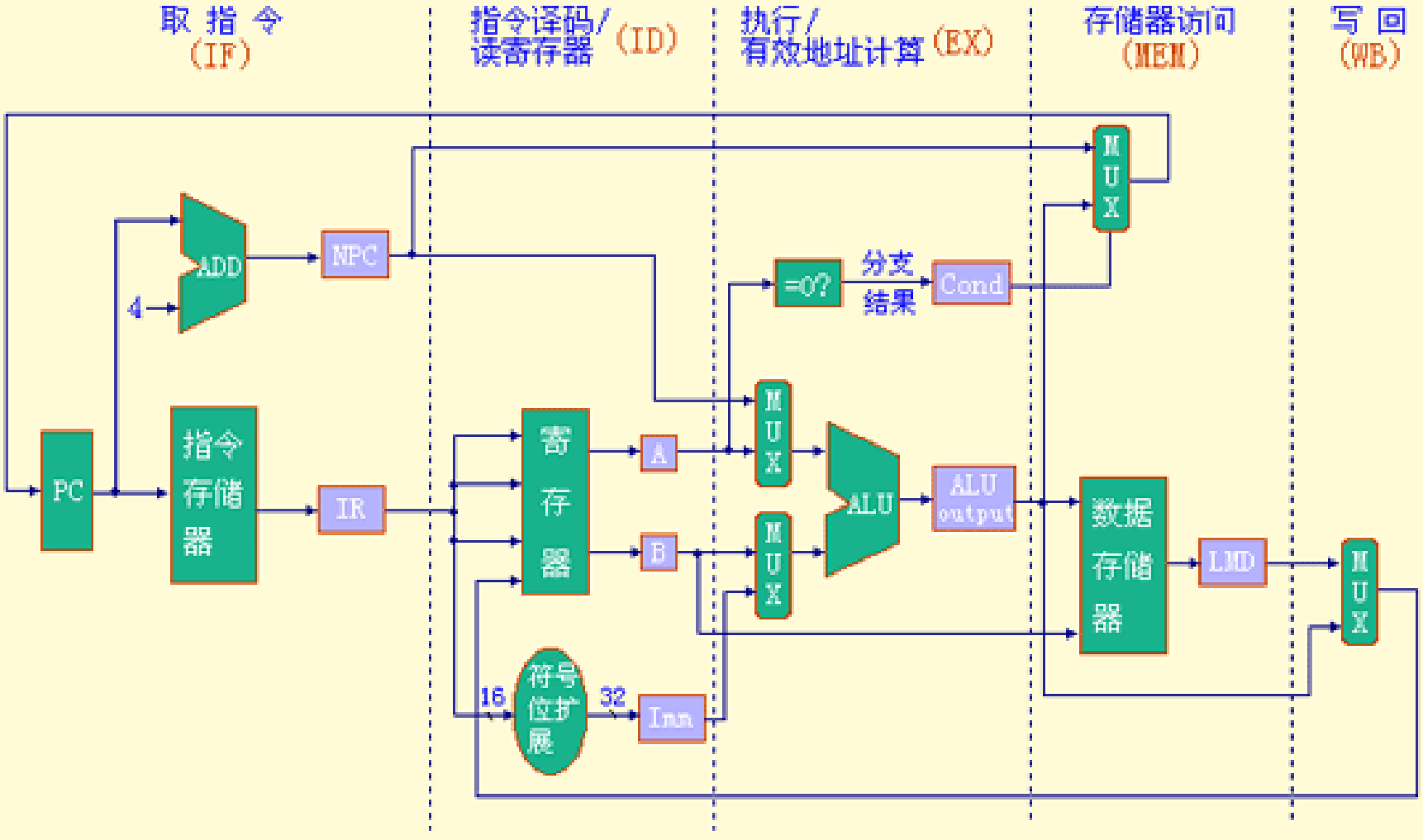
第六讲

计算机科学与技术学院
舒燕君

Recap

- MIPS指令集举例
  - ✓ MIPS指令集结构（寄存器、数据类型、寻址方式）
  - ✓ MIPS指令集的指令格式（I类、R类、J类）
  - ✓ MIPS指令集操作类型（Load和Store、ALU、分支和跳转、浮点）
- 简单MIPS CPU的实现
  - ✓ 指令系统的定义
  - ✓ 5个阶段的简单数据通路设计
  - ✓ 性能分析与改进

实现DLX指令的一种简单数据通路



第2章 指令系统

2.1 指令系统概述

2.2 指令系统结构的分类

2.3 寻址方式

2.4 操作数类型和大小

2.5 指令系统的设计与优化

2.6 指令系统的发展和改进

2.7 指令格式举例

2.8 MIPS的一种简单实现

2.3 寻址方式

- 在通用寄存器型指令集结构中，一般是利用寻址方式指明指令中的操作数是一个常数、一个寄存器操作数，或是一个存储器操作数。

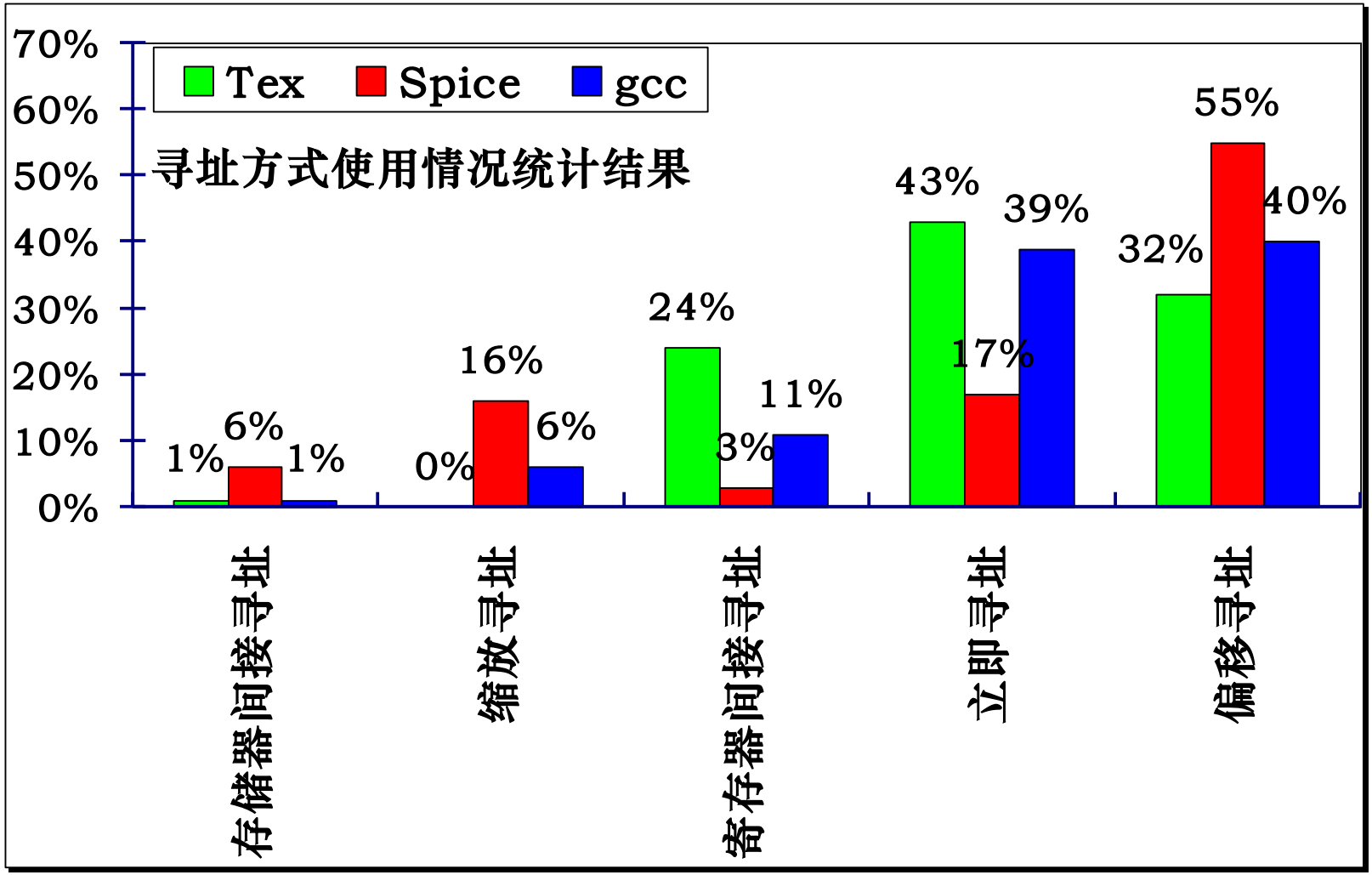
操作码	寻址方式	操作数	寻址方式	操作数

2.3 寻址方式

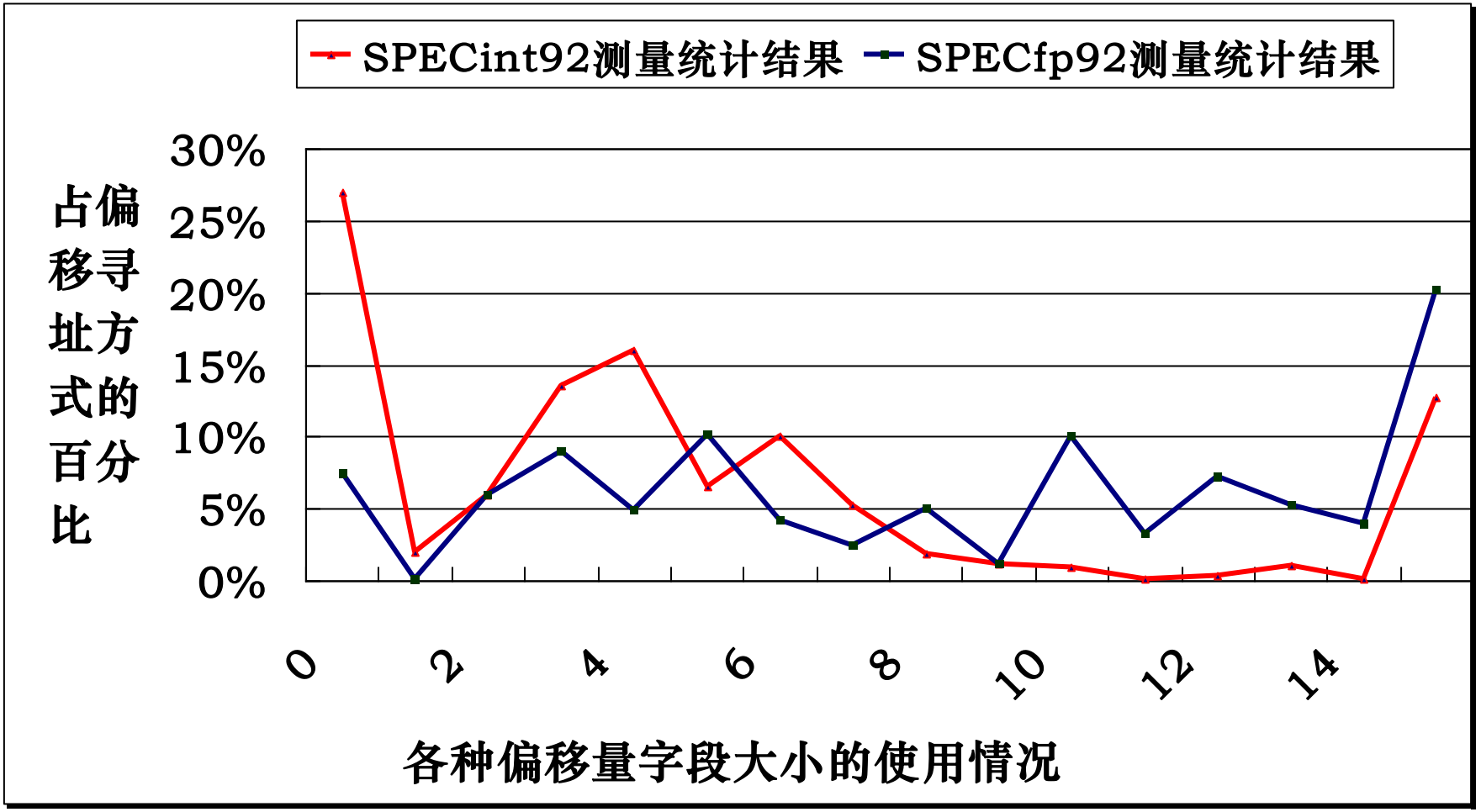
- 寻址实际上是从形式地址到实际地址的转换。形式地址由指令描述，实际地址也称为有效地址。
- 有效地址指明的是存储器单元的地址或寄存器地址。
- 必须加速有效地址生成。

寻址方式	指令实例	含　　义
寄存器寻址	ADD R1 , R2	Regs[R1]←Regs[R1] + Regs[R2]
立即值寻址	ADD R3 , #6	Regs[R3]←Regs[R3] + 6
偏移寻址	ADD R3 , 120(R2)	Regs[R3]←Regs[R3] + Mem[120+Regs[R2]]
寄存器间接寻址	ADD R4 , (R2)	Regs[R4]←Regs[R4] + Mem[Regs[R2]]
索引寻址	ADD R4 , (R2 + R3)	Regs[R4]←Regs[R4] + Mem[Regs[R2]+Regs[R3]]
直接寻址或绝对寻址	ADD R4 , (1010)	Regs[R4]←Regs[R4] + Mem[1010]
存储器间接寻址	ADD R2 , @(R4)	Regs[R2]←Regs[R2] + Mem[Mem[Regs[R4]]]
自增寻址	ADD R1 , (R2)+	Regs[R1]←Regs[R1] + Mem[Regs[R2]] Regs[R2]←Regs[R2] + d
自减寻址	ADD R1, -(R2)	Regs[R2]←Regs[R2] - d Regs[R1]←Regs[R1]+Mem[Regs[R2]]
缩放寻址	ADD R1 , 80(R2)[R3]	Regs[R1]←Regs[R1] + Mem[80 + Regs[R2] + Regs[R3]*d]

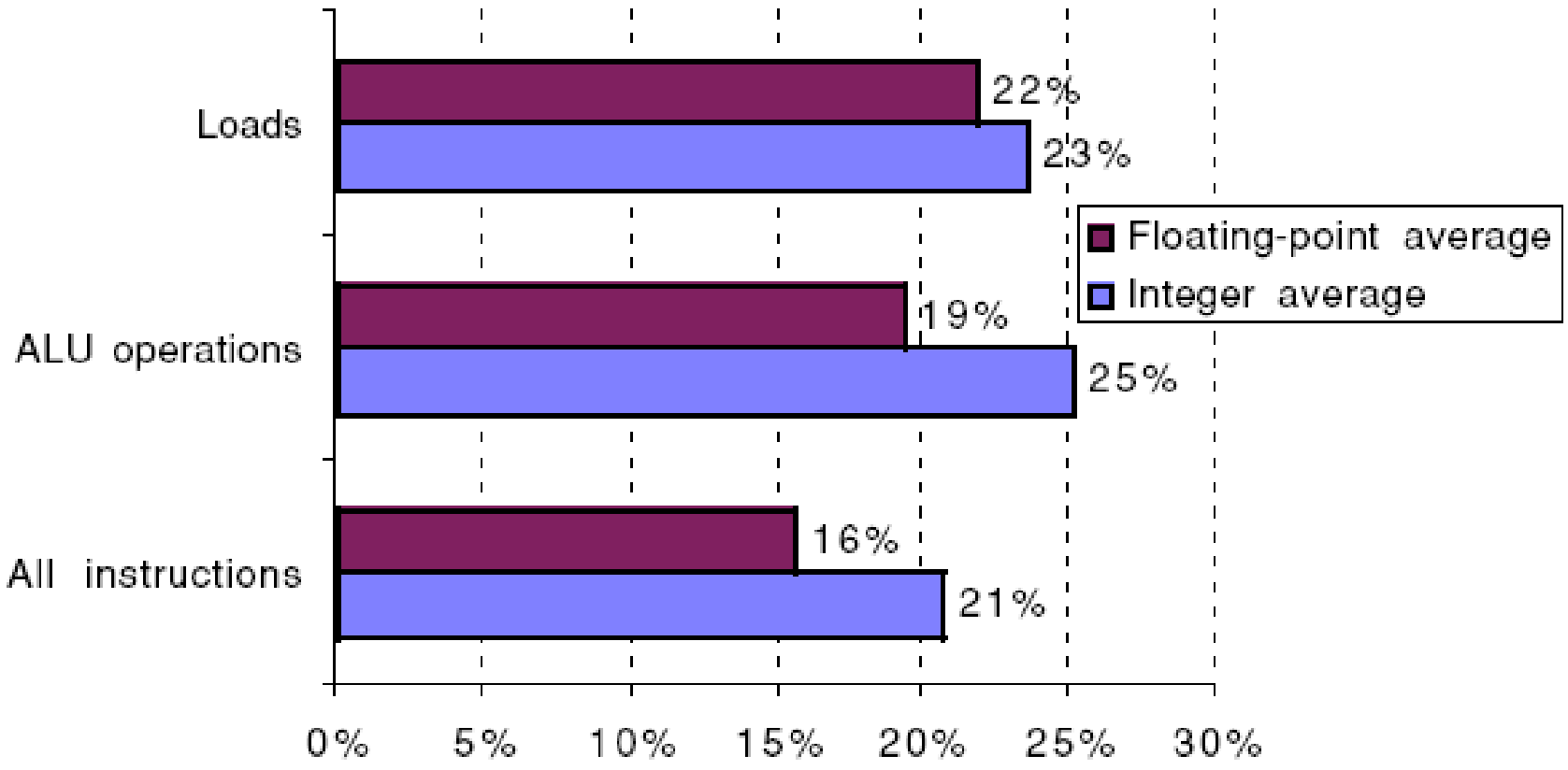
常用的一些操作数寻址方式



偏移寻址

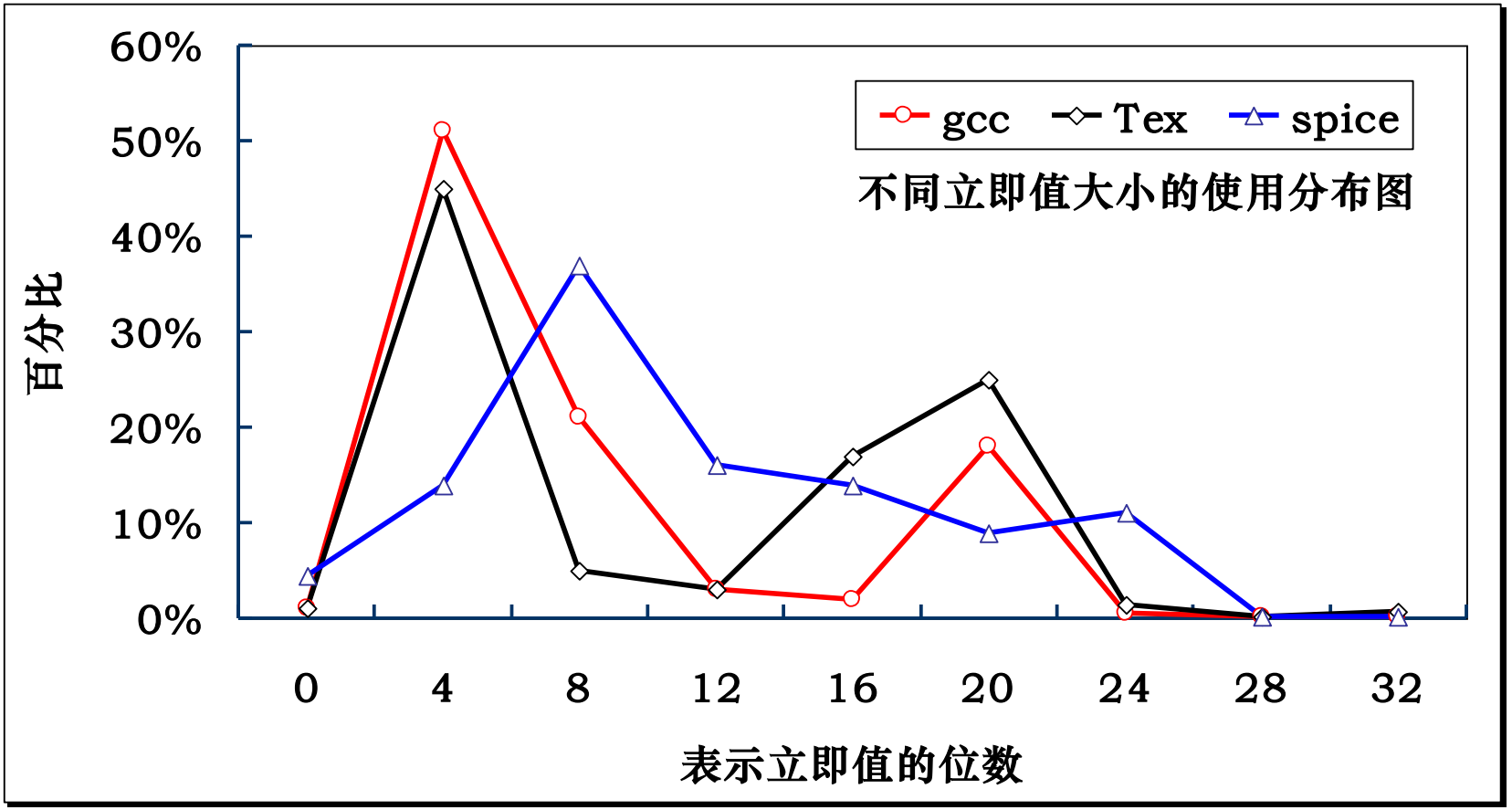

SPECint92测量统计结果
SPECfp92测量统计结果
占偏移寻址方式的百分比
30%
25%
20%
15%
10%
5%
0%
0
2
4
6
8
10
12
14
各种偏移量字段大小的使用情况

立即寻址



大约1/4的load指令和ALU指令采用了立即数寻址。

立即寻址


60%
50%
40%
30%
20%
10%
0%
百分比
gcc
Tex
spice
不同立即值大小的使用分布图
0
4
8
12
16
20
24
28
32
表示立即值的位数

2.3 寻址方式

- 两种表示寻址方式的方法
  - 将寻址方式编码于操作码中，由操作码描述相应操作的寻址方式。
  - 适合：处理机采用load-store结构，寻址方式只有很少几种。
    - ✓ MIPS指令集

2.3 寻址方式

- 两种表示寻址方式的方法
  - 在指令字中设置专门的寻址字段，用以直接指出寻址方式。
    - 灵活，操作码短，但需要设置专门的寻址方式字段，而且操作码和寻址方式字段合起来所需要的总位数可能会比隐含方法的总位数多。

适合：处理机具有多种寻址方式，且指令有多个操作数。

✓ VAX11中源操作数和目的操作数各有4位寻址方式位

2.3 寻址方式

- 一个需要注意的问题：物理地址空间的信息如何存放？

  如何在存储器中存放不同宽度的信息？

  以IBM370为例子进行讨论。

- 信息有字节、半字（双字节）、单字（4字节）和双字（8字节）等宽度。
- 主存宽度为8个字节。采用按字节编址，各类信息都是用该信息的首字节地址来寻址。
- 允许它们任意存储
- 很可能会出现一个信息跨存储字边界而存储于两个存储单元中

2.3 寻址方式

- 信息宽度不超过主存宽度的信息必须存放在一个存储字内，不能跨边界。
  - 必须做到：信息在主存中存放的起始地址必须是该信息宽度（字节数）的整数倍

    信息存储的整数边界概念
  - 满足以下条件
    - 字节信息的起始地址为：×…××××
    - 半字信息的起始地址为：×…×××0
    - 单字信息的起始地址为：×…××00
    - 双字信息的起始地址为：×…×000
  - 存在存储空间的浪费，但保证访问速度。

2.3 寻址方式

- 对s字节的对象访问地址为A，如果A mod s =0 称为边界对齐。
- 边界对齐的原因是存储器本身读写的要求，存储器本身读写通常就是边界对齐的，对于不是边界对齐的对象的访问可能要导致存储器的两次访问，然后再拼接出所需要的数（或发生异常）。

Address mod 8	0	1	2	3	4	5	6	7
Byte	Aligned	Aligned	Aligned	Aligned	Aligned	Aligned	Aligned	Aligned
2 Bytes	Aligned		Aligned		Aligned		Aligned	
2 Bytes		Misaligned		Misaligned		Misaligned		Misalign
4 Bytes	Aligned				Aligned			
4 Bytes		Misaligned				Misaligned		
4 Bytes			Misaligned				Misaligned	
4 Bytes				Misaligned				Misalign
8 Bytes	Aligned							
8 Bytes		Misaligned						

第2章 指令系统

2.1 指令系统概述

2.2 指令系统结构的分类

2.3 寻址方式

2.4 操作数类型和大小

2.5 指令系统的设计与优化

2.6 指令系统的发展和改进

2.7 指令格式举例

2.8 MIPS的一种简单实现

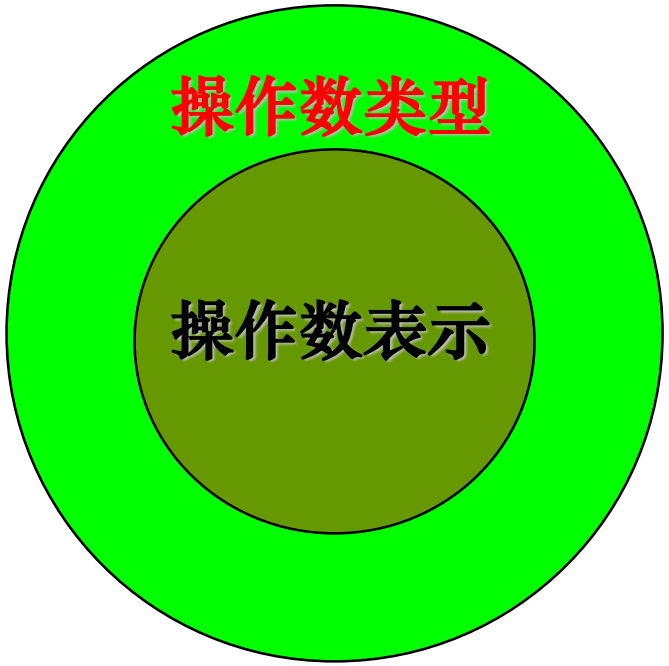
2.4 操作数类型和大小

一、操作数类型

- 操作数类型和操作数表示也是软硬件主要界面之一。
- 操作数类型是面向应用、面向软件系统所处理的各种数据结构。
- 操作数表示是硬件结构能够识别、指令系统可以直接调用的那些结构。

一、操作数类型

- 操作数表示所表征的那些操作数类型，是应用软件和系统软件所处理的操作数类型的子集。



一、操作数类型

- 确定操作数表示实际上也是软硬件取舍折衷的问题
  - 计算机即使只具有最简单的操作数表示，如只有整数（定点）表示法，也可以通过软件方法处理各种复杂的操作数类型，但是这样会大大降低系统的效率。
  - 如果各种复杂的操作数类型均包含在操作数表示之中，无疑会大大提高系统的效率，但是所花费的硬件代价也很高。

一、操作数类型

- 整数（定点）：二进制补码表示；其大小可以是字节（8位）、半字（16位）或单字（32位）。
- 浮点：可以分为单精度浮点（单字大小）和双精度浮点（双字大小）。当前普遍采用的是IEEE 754浮点操作数表示标准。
- 字符和字符串：8位ASCII码表示。

一、操作数类型

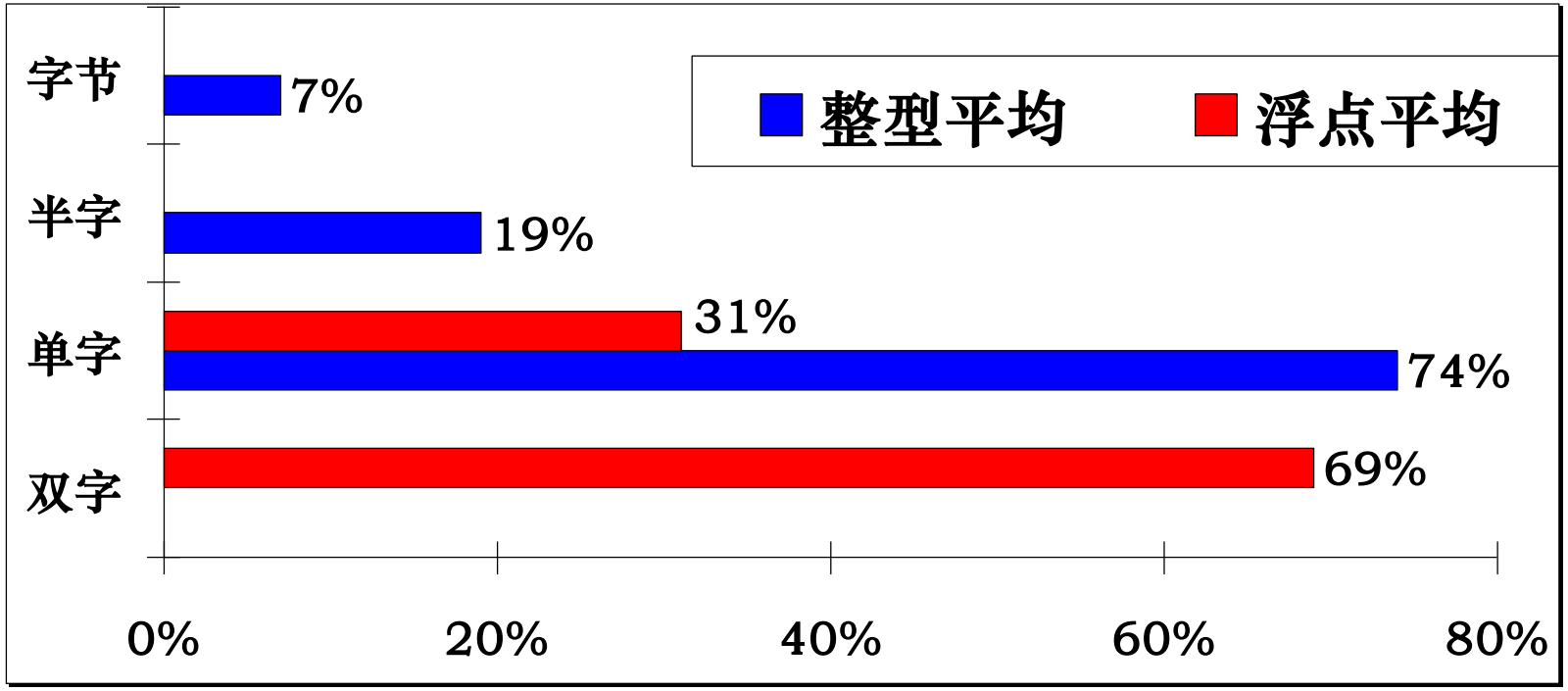
- 十进制：面向商业应用，通常采用“压缩十进制”或“二进制编码十进制（BCD）”表示。压缩十进制数据表示用4位编码数字0～9，然后将两个十进制数字压缩在一个字节中存储。如果将十进制数字直接用字符串来表示，就叫做“非压缩十进制”表示法。

一、操作数类型

- 操作数类型的表示主要有如下两种方法：
  - 操作数的类型可以由操作码的编码指定，这也是最常见的一种方法；
  - 数据可以附上由硬件解释的标记（tag），由这些标记指定操作数的类型，从而选择适当的运算。然而有标记数据的机器却非常少见。

二、操作数大小

- 一般的操作数类型大小选择主要有：字节、半字（16位）、单字（32位）、和双字（64位）。



第2章 指令系统

2.1 指令系统概述

2.2 指令系统结构的分类

2.3 寻址方式

2.4 操作数类型和大小

2.5 指令系统的设计与优化

2.6 指令系统的发展和改进

2.7 指令格式举例

2.8 MIPS的一种简单实现

2.5 指令系统的设计和优化

2.5.1 指令系统设计的基本原则

2.5.2 控制指令

2.5.3 指令操作码的优化

2.5 指令系统的设计和优化

2.5.1 指令系统设计的基本原则

2.5.2 控制指令

2.5.3 指令操作码的优化

2.5.1 指令系统设计的基本原则

1. 指令系统的设计

- 首先考虑所应实现的基本功能，确定哪些基本功能应该由硬件实现，哪些功能由软件实现比较合适。
- 包括
  - 指令的功能设计
  - 指令格式的设计

2.5.1 指令系统设计的基本原则

1. 指令系统的设计

– 一种指令系统中的指令到底要支持哪些类型的操作呢？这就是所谓的指令系统功能设计问题。

2.5.1 指令系统设计的基本原则

2. 在确定哪些基本功能用硬件来实现时，主要考虑3个因素：速度、成本、灵活性。

- 硬件实现的特点

  速度快、成本高、灵活性差

- 软件实现的特点

  速度慢、价格便宜、灵活性好

3. 对指令系统的基本要求

完整性、规整性、正交性、高效率、兼容性

2.5.1 指令系统设计的基本原则

- 完整性：在一个有限可用的存储空间内，对于任何可解的问题，编制计算程序时，指令系统所提供的指令足够使用。
  - 要求指令系统功能齐全、使用方便
  - 下表为许多指令系统结构都包含的一些指令类型
    - 前4类属于通用计算机系统的基本指令
    - 对于最后4种类型的操作，不同指令系统结构的支持大不相同 。

指令集操作的分类

算术和逻辑运算	整数的算术和逻辑操作：加、减、与、或等
数据传输	存数/取数
控制	分支、跳转、过程调用和返回、自陷等
系统	操作系统调用、虚拟存储器管理等
浮点	浮点操作：加、乘等
十进制	十进制加、十进制乘、十进制到字符的转换
字符串	字符串移动、字符串比较、字符串搜索等
图形	像素操作、压缩/解压操作等

2.5.1 指令系统设计的基本原则

- 规整性：主要包括对称性和均匀性。
  - 对称性：所有与指令系统有关的存储单元的使用、操作码的设置等都是对称的。
    - ✓例如，在存储单元的使用上，所有通用寄存器都要同等对待。在操作码的设计上，如果设置了**A-B**的指令，就应该也设置**B-A**的指令。
  - 均匀性：指对于各种不同的操作数类型、字长、操作种类和数据存储单元，指令的设置都要同等对待。
    - ✓例如，如果某机器有**5**种数据表示，**4**种字长，两种存储单元，则要设置**5*4*2=40**种同一操作的指令（如加法指令）。

2.5.1 指令系统设计的基本原则

- 正交性：在指令中各个不同含义的字段，如操作类型、数据类型、寻址方式字段等，在编码时应互不相关、相互独立。
- 高效率：指指令的执行速度快、使用频度高。
- 兼容性：主要是要实现向后兼容，指令系统可以增加新指令，但不能删除指令或更改指令的功能。

2.5 指令系统的设计和优化

2.5.1 指令系统设计的基本原则

2.5.2 控制指令

2.5.3 指令操作码的优化

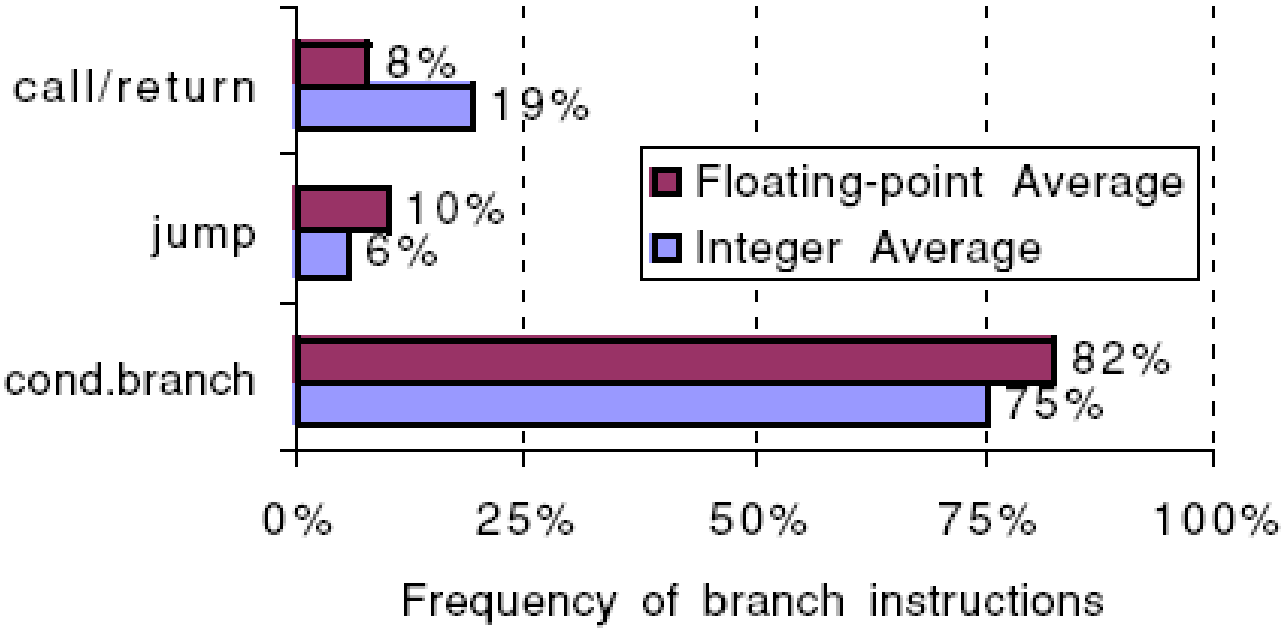
2.5.2 控制指令

- “跳转”（Jump）：当控制指令为无条件改变控制流时，我们称之为“跳转”。
- “分支”（Branch）：而当控制指令是有条件改变控制流时，我们称之为“分支”。

程序中控制流程的改变情况包括：

- 跳转（jump）；
- 条件分支（conditional branch）；
- 过程调用（call）；
- 过程返回（return）。

控制指令的使用频率

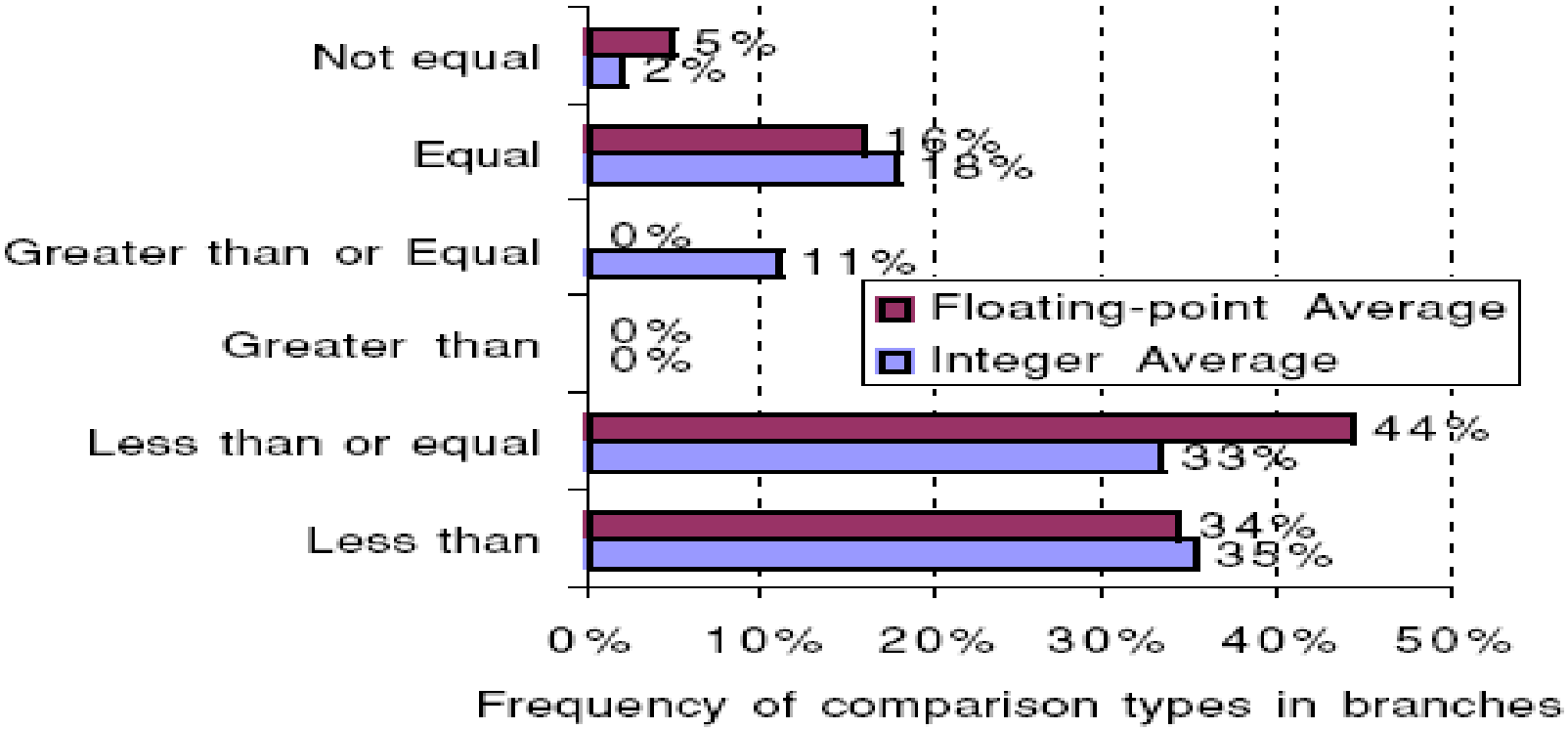


改变控制流的大部分指令是分支指令（条件转移）。

条件分支指令的表示

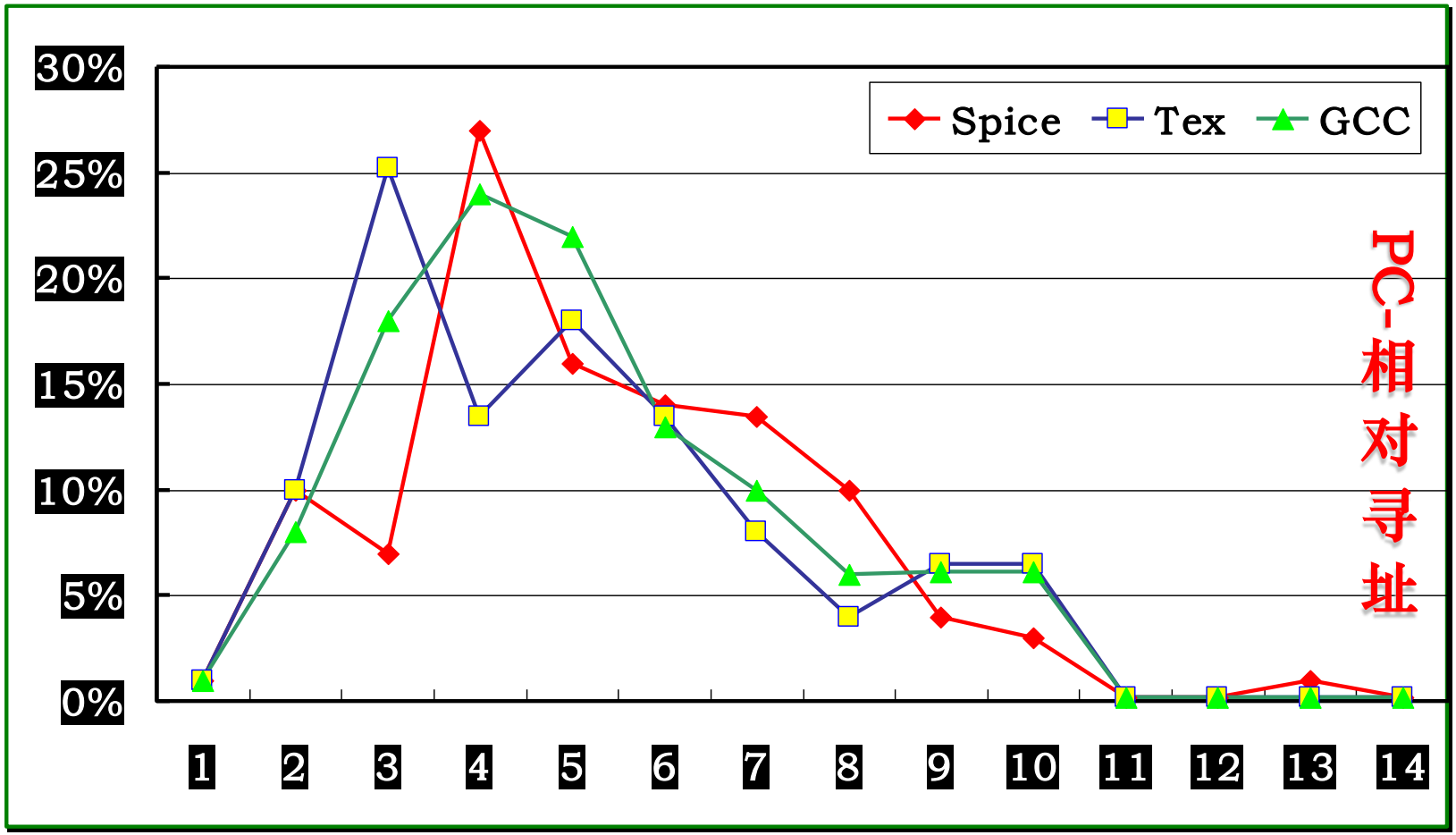
分支条件表示	优　点	缺　点
条件码（CC）：在程序的控制下，由ALU操作设置特殊的位	可以自由设置分支条件	必须从一条指令将分支条件信息传送到分支指令，所以CC是额外状态，条件码限制了指令执行顺序
条件寄存器：比较指令把比较结果放入任何一个寄存器，检测时就检测该寄存器	简单	占用了一个寄存器
比较分支：比较操作是分支指令的一部分，比较受限制	一条指令完成了两条指令的功能	分支指令的操作增多

分支比较类型



小于或等于分支占主导地位。

分支目标地址的表示

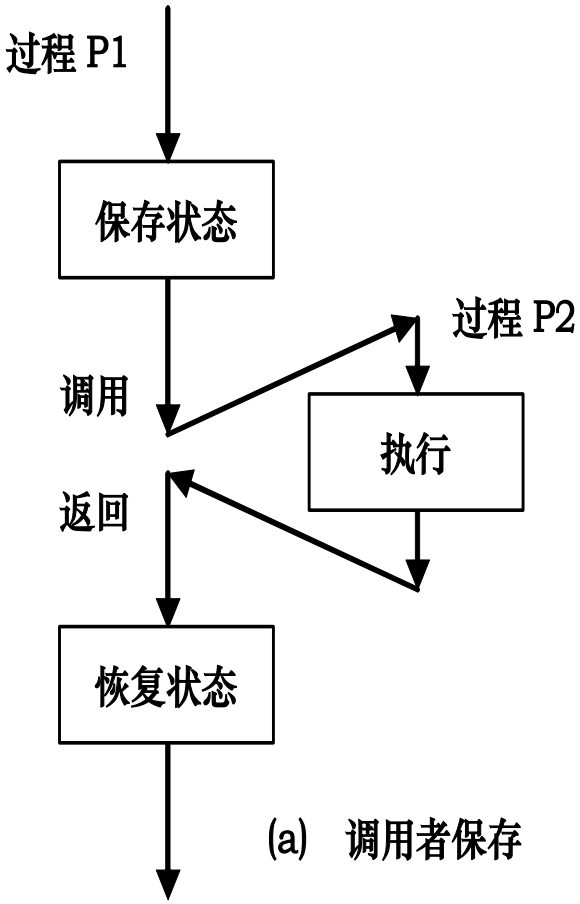


– 采用4～8位的偏移量字段（以指令字为单位）就能表示大多数控制指令的转移目标地址了

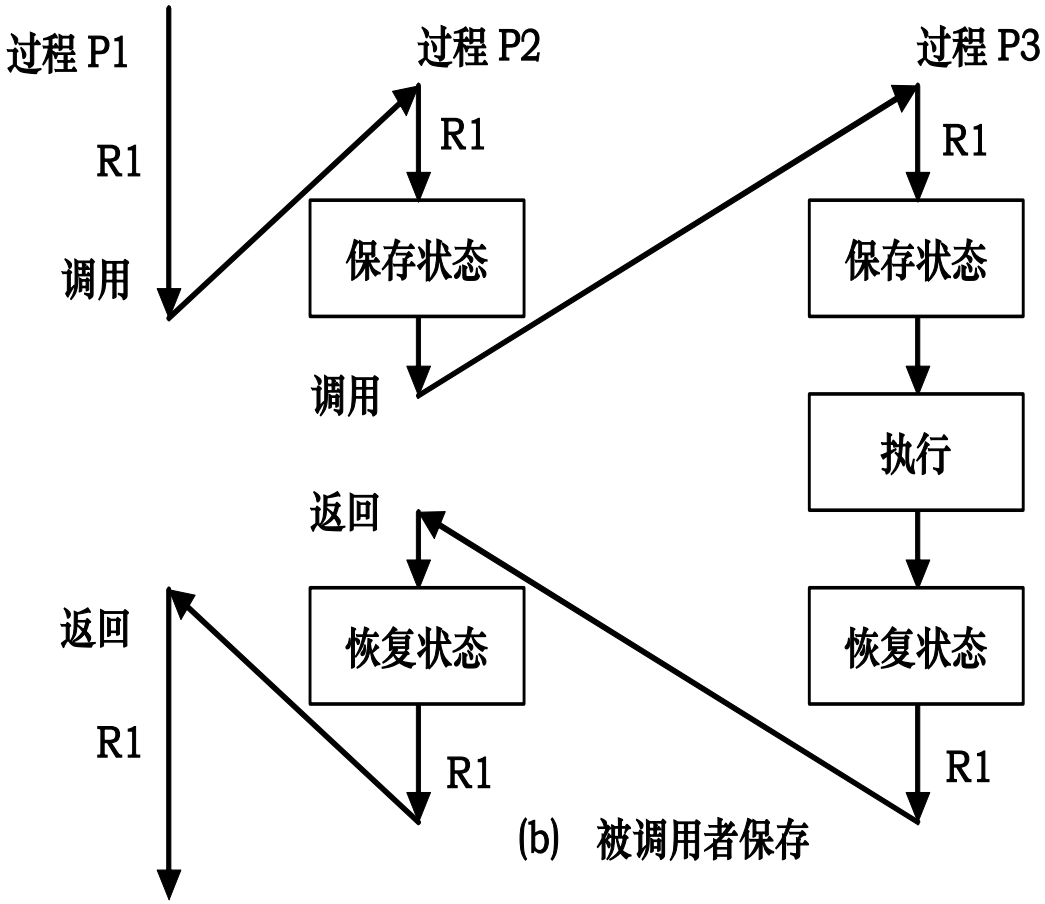
过程调用和返回的状态保存

- “调用者保存”（caller saving）方法：如果采用调用者保存策略，那么在一个调用者调用别的过程时，必须保存调用者所要保存的寄存器，以备调用结束返回后，能够再次访问调用者。
- “被调用者保存”（callee saving）方法：如果采用被调用者保存策略，那么被调用的过程必须保存它要用的寄存器，保证不会破坏过程调用者的程序执行环境，并在过程调用结束返回时，恢复这些寄存器的内容。

两种保存策略的比较



(a) 调用者保存



(b) 被调用者保存

2.5 指令系统的设计和优化

2.5.1 指令系统设计的基本原则

2.5.2 控制指令

2.5.3 指令操作码的优化

2.5.3 指令操作码的优化

- 指令由两部分组成：操作码、地址码
- 指令格式的设计

  确定指令字的编码方式，包括操作码字段和地址码字段的编码和表示方式。
- 指令格式的优化：如何用最短的位数来表示指令的操作信息和地址信息。

2.5.3 指令操作码的优化

1. 等长扩展码

– 为了便于分级译码，一般都采用等长扩展码。（在早期的计算机上）

例如：15/15/15法和8/64/512法

- 选用哪种编码法取决于指令使用频度p_i的分布。若在头15种指令中p_i的值都比较大，但在后30种指令后急剧减少，则应选择15/15/15法；若p_i的值在头8种指令中较大，之后的64种指令的p_i值也不太低，则应选择8/64/512法。
- 衡量标准：看哪种编码法能使平均码长最短。

2.5.3 指令操作码的优化

15 { 0000, 0001, ⋮, 1110 }

15 { 1111 0000, 1111 0001, ⋮, 1111 1110 }

15 { 1111 1111 0000, 1111 1111 0001, ⋮, 1111 1111 1110 }

15/15/15 编码法

8 { [0] 000, [0] 001, ⋮, [0] 111 }

64 { [1] 000 [0] 000, [1] 000 [0] 001, ⋮, [1] 111 [0] 111 }

512 { [1] 000 [1] 000 [0] 000, [1] 000 [1] 000 [0] 001, ⋮, [1] 111 [1] 111 [0] 111 }

8/64/512 编码法

2.5.3 指令操作码的优化

2. 定长操作码

- 固定长度的操作码：所有指令的操作码都是同一的长度（如8位）。

  许多计算机都采用（特别是RISC结构的计算机）
- 保证操作码的译码速度、减少译码的复杂度。
- 以程序的存储空间为代价来换取硬件实现上的好处。

第2章 指令系统

2.1 指令系统概述

2.2 指令系统结构的分类

2.3 寻址方式

2.4 操作数类型和大小

2.5 指令系统的设计与优化

2.6 指令系统的发展和改进

2.7 指令格式举例

2.6 指令系统的发展和改进

- 一个方向是强化指令功能，实现软件功能向硬件功能转移，基于这种指令集结构而设计实现的计算机系统称为复杂指令集计算机（CISC）。
- 八十年代发展起来的精简指令集计算机（RISC），其目的是尽可能地降低指令集结构的复杂性，以达到简化实现，提高性能的目的。

CISC指令集功能设计

1 面向目标程序增强指令功能

- 对大量的目标程序及其执行情况进行统计分析，找出那些使用频度高、执行时间长的指令或指令串。对于使用频度高的指令，用硬件加快其执行；对于使用频度高的指令串，用一条新的指令来替代。
- 既能减少目标程序的执行时间，也能有效地缩短程序的长度。
- 可以从以下几个方面来改进：

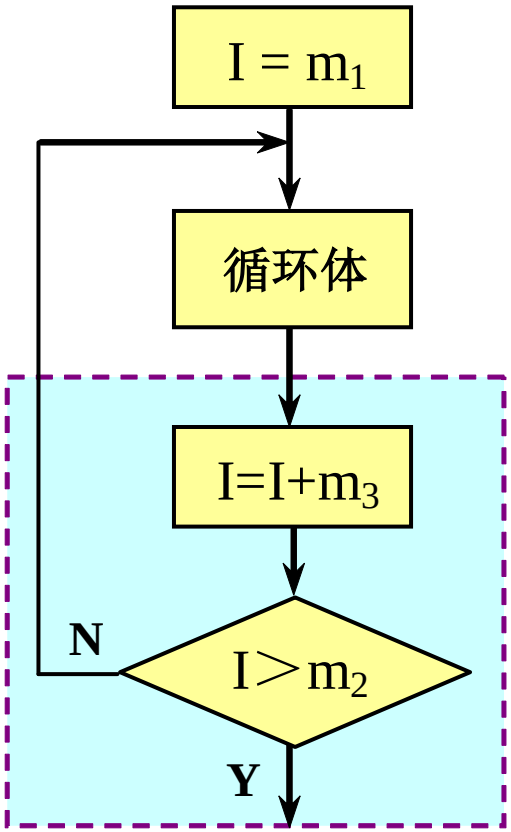
  (1) 增强运算型指令的功能

  (2) 增强数据传送指令的功能

  (3) 增强程序控制指令的功能

例如：循环在程序中占有相当大的比例，所以在指令上提供专门的支持。

- 循环控制占有较大比例
- 循环控制部分通常用3条指令完成：
  - 一条加法指令
  - 一条比较指令
  - 一条分支指令
- 设置循环控制指令，用一条指令完成上述3条指令的功能。



一般循环程序的结构

CISC指令集功能设计

2 面向高级语言的优化实现来改进指令系统

（缩小高级语言与机器语言的语义差距）

高级语言与一般的机器语言的语义差距非常大，为高级语言程序的编译带来了一些问题。

- ✓ 编译器本身比较复杂；
- ✓ 编译生成的目标代码比较难以达到很好的优化。

CISC指令集功能设计

(1) 增强对高级语言和编译器的支持

- ✓ 对高级语言中使用频度高、执行时间长的语句，增强有关指令的功能，加快这些指令的执行速度，或者增加专门的指令，可以达到减少目标程序的执行时间和减少目标程序长度的目的。
- ✓ 增强系统结构的规整性，减少系统结构中的各种例外情况。

CISC指令集功能设计

(2) 高级语言计算机

① 间接执行高级语言机器

高级语言作为机器的汇编语言。

② 直接执行高级语言的机器

直接把高级语言作为机器语言。

（一种比较激进的方法）

采用“比较简单的系统结构＋软件”的做法能够在较低成本和复杂度的前提下，提供更高的性能和灵活性。

CISC指令集功能设计

3 面向操作系统的优化实现改进指令系统

(1) 处理机工作状态和访问方式的切换；

(2) 进程的管理和切换；

(3) 存储管理和信息保护；

(4) 进程的同步与互斥，信号的管理等。

RISC指令集功能设计

- CISC结构存在着如下缺点：

  1. 在CISC结构的指令系统中，各种指令的使用频率相差悬殊。据统计，有20％的指令使用频率最大，占运行时间的80％。也就是说，有80％的指令在20％的运行时间内才会用到。

RISC指令集功能设计

2. CISC结构指令系统的复杂性带来了计算机体系结构的复杂性。大量占用芯片面积，给VLSI设计造成很大困难。这不仅增加了研制时间和成本，而且还容易造成设计错误。
3. CISC结构的指令系统中，许多复杂指令需要很复杂的操作，因而运行速度慢。
4. 在CISC结构的指令系统中，由于各条指令的功能不均衡性，不利于采用先进的计算机体系结构技术（如流水技术）来提高系统的性能。

RISC指令集功能设计

执行频率排序	80X86指令	指令执行频率
1	Load	22%
2	条件分支	20%
3	比较	16%
4	Store	12%
5	加	8%
6	与	6%
7	减	5%
8	寄存器－寄存器间数据移动	4%
9	调用	1%
10	返回	1%
合　计		95%

RISC指令集功能设计

- 进行RISC计算机指令集结构的功能设计时，我们并不能简单地着眼于精简指令系统上，更重要的目的是使得计算机体系结构更加简单、更加合理和更加有效，克服CISC结构的缺点，使机器速度更快，程序运行时间缩短，从而提高计算机系统的性能。

RISC指令集功能设计原则

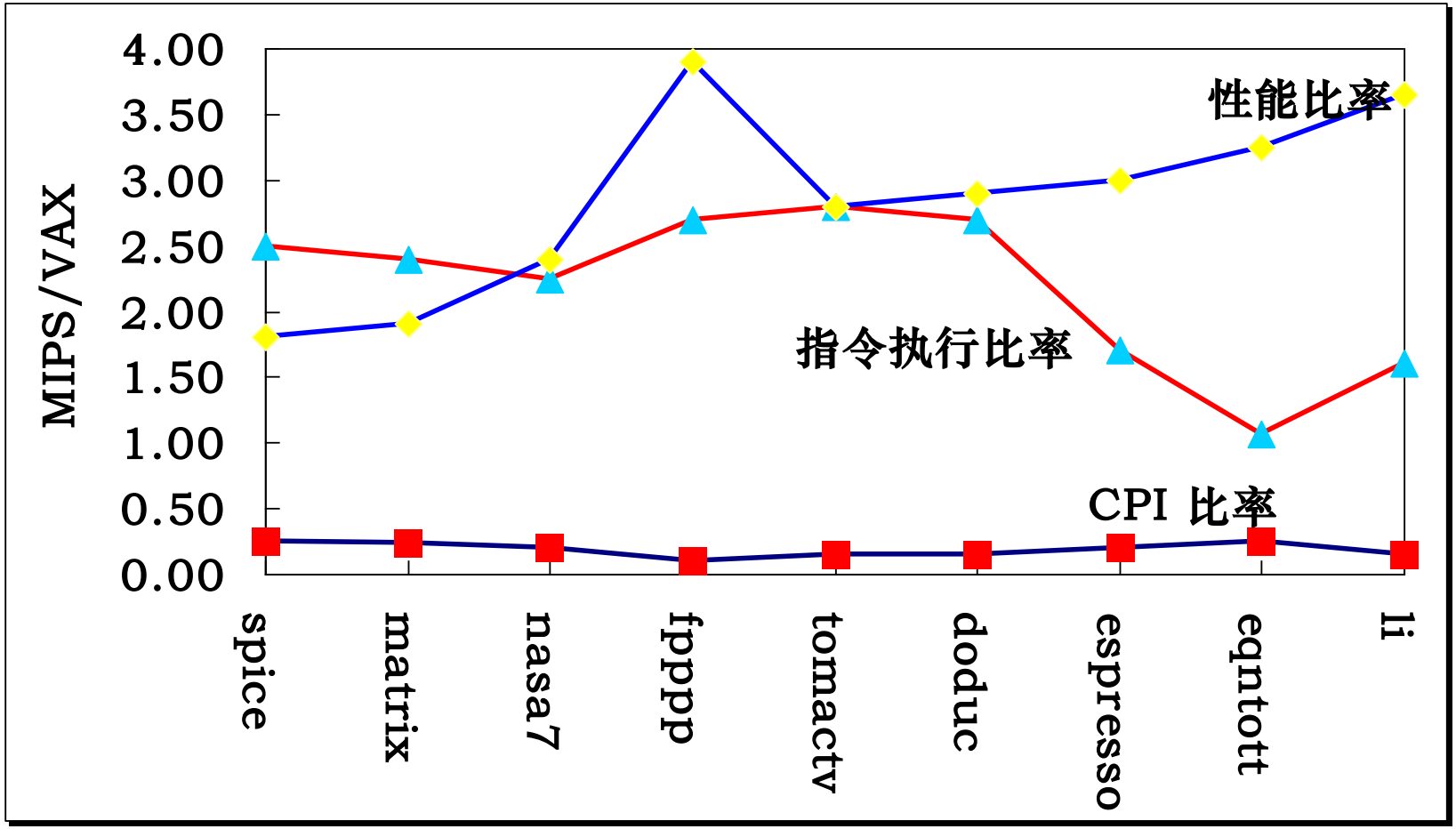
- 选取使用频率最高的指令，并补充一些最有用的指令；
- 每条指令的功能应尽可能简单，并在一个机器周期内完成（采用流水线技术后）；
- 所有指令长度均相同；
- 只有load和store操作指令才访问存储器，其它指令操作均在寄存器之间进行；
- 大多数指令都采用硬连线技术；
- 以简单有效的方式支持高级语言；
- 充分利用流水线技术。

RISC的效能分析（MIPS）

- 问题的提出：
  - MIPS指令集结构的指令格式、寻址方式和操作都非常简单。也许有人会担心，这些特性会使得目标代码中指令条数增多，导致程序运行时间加长，从而使这种指令集结构的机器性能并不会太高。

$$T_{CPU} = IC \times CPI \times T_{CLK}$$

MIPS的效能分析


性能比率
指令执行比率
CPI 比率
MIPS/VAX
4.00
3.50
3.00
2.50
2.00
1.50
1.00
0.50
0.00
spice
matrix
nasa7
fpppp
tomactv
doduc
espresso
eqntott
li

本章小结

- 讨论指令系统的基本概念
- 指令系统结构的分类
  - 堆栈型、累加器型、通用寄存器型
  - 根据**ALU**操作数来源，通用寄存器型包括：寄存器-存储器型和寄存器-寄存器型
- 寻址方式
  - 操作数的寻址
  - 偏移寻址与立即寻址的范围

本章小结

- 操作数的分类和大小
- 指令系统设计的设计和优化
  - 基本原则（完整性、规整性、正交性、高效率、兼容性）
  - 控制指令、操作码的优化、指令集编码
- 指令系统的发展和优化
- **MIPS**指令集
- 简单**MIPS CPU**的设计

第二章作业

- 王志英教材，P62，T2，T12，T16
- 和第一章作业一起交（9月20日）
- 地点：综合楼514

课程报告

以下报告内容均需要引用本课程的相关内容：

- 选择你认为计算机发展历史上最重要的十台机器，并说明理由。
- 由于AI的广泛应用，CPU+GPU的架构越来越普遍，比较各厂家架构设计上的特点。
- 近五年来，在CPU、存储、总线、IO 等方面有什么重要的新进展，分析它的影响和未来的发展。